

HGL: Accelerating Heterogeneous GNN Training with Holistic Representation and Optimization

Yuntao Gui[†], Yidi Wu[†], Han Yang[†], Tatiana Jin[†], Boyang Li[†], Qihui Zhou[†], James Cheng[†], Fan Yu[‡]

[†]The Chinese University of Hong Kong, Hong Kong SAR, China

{ytgui,ydwu,hyang,tjin,byli,qhzhou,jcheng}@cse.cuhk.edu.hk

[‡]Huawei Technologies Co. Ltd, Shenzhen, China

fan.yu@huawei.com

Abstract—Graph neural networks (GNNs) have shown to significantly improve graph analytics. Existing systems for GNN training are primarily designed for homogeneous graphs. In industry, however, most graphs are actually heterogeneous in nature (i.e., having multiple types of nodes and edges). Existing systems train a heterogeneous GNN (HetGNN) as a composition of homogeneous GNN (HomoGNN) and thus suffer from critical limitations such as lack of memory optimization and limited operator parallelism. To address these limitations, we propose HGL — a heterogeneity-aware system for GNN training. At the core of HGL is an intermediate representation, called HIR, which provides a holistic representation for GNNs and enables cross-relation optimization in HetGNN training. We devise tailored optimizations on HIR, including graph stitching, operator fusion and operator bundling. Compared with DGL and PyG, HGL achieves a speedup from 7 to 22 times for training HetGNNs.

Index Terms—Graph Neural Networks, Heterogeneous Graphs, Deep Learning Systems

I. INTRODUCTION

In recent years, a significant amount of efforts from both academia and industry have been invested in applying various machine learning techniques on graph-structured data to gain valuable insights for real-world applications. Among them, GNNs have received increasing attention because of their success in many fundamental graph analytic tasks such as link prediction and node classification.

Earlier success of GNNs (e.g., GCN [1], GAT [2], GraphSAGE [3]) are mostly concentrated on *homogeneous graphs* (HomoGs), i.e., graphs with only one type of nodes and edges. However, most graphs in real-world applications are naturally *heterogeneous graphs* (HetGs), i.e., graph with multiple types of nodes and/or edges. Compared with HomoGs, HetGs can provide much richer information for many important applications such as recommendation and risk management, enabling machine learning models to capture more intelligent information and achieve much higher accuracy. For example, many recently proposed *heterogeneous GNN models* (HetGNNs) such as [4]–[7] have demonstrated their great ability to capture and leverage the previously unexploited heterogeneous information in HetGs, thereby exceeding the performance of HomoGs by a large margin.

A HOMO GNN (e.g., GAT) takes node-wise input feature tensor as input and produces node embedding by feeding the tensor to a stack of graph convolution layers (e.g., GATConv).

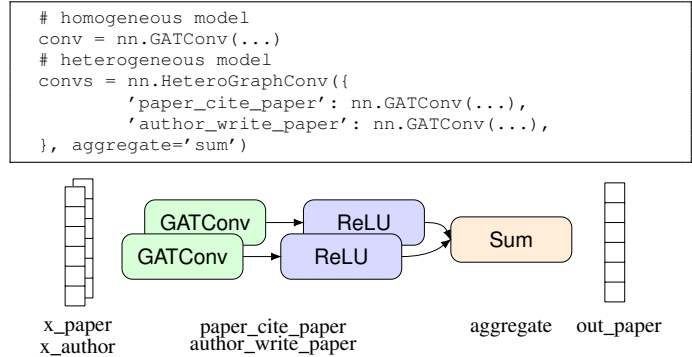


Fig. 1: R-GAT as a composition of GATs

Each graph convolution layer generates messages using common neural network operators (e.g., a fully connected layer), passes the messages through edges, and aggregates them at destination nodes. However, such HOMO GNNs are not suitable for HetGs as they will discard the rich information contained in the relation types and node types. To support GNNs on HetGs, GNN training systems can view a HetGNN as a composition of multiple HOMO GNNs constructed on the homogeneous subgraphs decomposed from the HetG. We illustrate using a representative HetGNN, i.e., R-GAT (a heterogeneous port of homogeneous GAT) [8], [9], in Fig. 1. A heterogeneous graph convolution layer starts with a *relation decomposition* that disintegrates the original HetG into multiple subgraphs following user-defined relations (defined based on meta-paths such as *author_write_paper* and *paper_cite_paper*). For each relation, homogeneous graph convolution layers (e.g., GATConv) are used to compute embeddings at each destination node (e.g., *paper*). The heterogeneous GNN embedding (e.g., *out_paper*) of each destination node is constructed by aggregating embeddings from all homogeneous GNN layers of different relations. Such heterogeneous GNN layers are stacked and computed iteratively to form a complete HetGNN model.

Currently, existing GNN training systems have no native support for HetGNNs. They construct a HetGNN model through composition of HOMO GNNs using the existing HOMO GNN API, as the example shown in Fig. 1. Then, they perform HetGNN training by simply reusing HOMO GNN training procedures multiple times, which imposes critical limita-

tions on computation efficiency. Although many optimization techniques for HomoGNN training have been proposed [10], [11], important optimization opportunities that are specific to HetGNN training are neglected in existing works. We identify two key characteristics of HetGNNs compared with HomoGNNs and the corresponding key implications for performance optimizations as follows:

- **C1: HetGNNs require more memory optimization.** For HetGNNs, the number of graph convolution layers, as well as the memory consumption of the intermediate results generated during the execution of graph convolution, are proportional to the number of relations in a HetG. In many real-world HetGs, the number of relations can be up to hundreds or more. Allocating memory separately for each subgraph (corresponding to each relation) causes fragmented memory allocation and can produce many small tensors due to the imbalance in the sizes of the subgraphs for different relations. This results in excessive memory consumption and slow execution, which lead to opportunities for memory optimization.
- **C2: HetGNNs expose more computational parallelism.** A heterogeneous graph convolution layer consists of multiple homogeneous layers (e.g., the GATConv layers taking x_{author} and x_{paper} as input). Thus, the amount of computation grows linearly with the number of relations, which exposes more parallel execution opportunities.

Unfortunately, exploiting the aforementioned memory and computational characteristics in HetGNN training is non-trivial. Firstly, since the execution in current GNN systems is unaware of graph heterogeneity, the minimum unit of computation and memory allocation is a HomoGNN. This prohibits any fine-grained optimization of memory allocation and parallel execution. Secondly, existing systems provide graph related calculation (such as graph convolution) by abstracting message passing based on operators in deep learning (DL) frameworks. Thus, the functionalities provided to users are limited to piecing together different computations on top of DL computational flow. The entire computational dataflow is discontinuous, which makes holistic optimizations for HetGNN training difficult, if not impossible.

We propose *HGL* — a GNN system that takes HetGNN models as first-class citizens. Since a HetGNN model consists of decomposed relations and HomoGNN computations, a HetGNN-native system should (1) express HomoGNNs and HetGNNs in a unified framework (preferably not to burden users with new APIs), (2) handle cross-relation computations efficiently, and (3) support holistic optimizations for HetGNN training. We design an intermediate representation, called *HIR*, to achieve these goals. To reuse existing libraries (e.g., convolution layers such as GCNConv and GATConv), *HGL* is designed to be compatible with existing message passing programming models. Given a GNN model (HetGNN or HomoGNN) constructed by users, *HGL* analyzes its implementation and translates it into *HIR*. We also propose tailored optimizations on *HIR*, including *graph stitching*, *operator*

fusion and *operator bundling*, which address the memory fragmentation and limited parallelism problems mentioned in C1 and C2 above. Compared with DGL and PyG, *HGL* achieves $7\times$ to $22\times$ speedup in throughput for HetGNN training. Compared with DGL, *HGL* only consumes 14.4%-29.1% memory, reduces small memory blocks by a maximum of 92%, and doubles the GPU utilization for HetGNN training.

We summarize our contributions as follows:

- We propose a holistic intermediate representation, *HIR*, for both HomoGNNs and HetGNNs.
- We develop systematic optimization techniques on *HIR* for both HomoGNN and HetGNN training.
- We validate by experiments that *HGL* effectively addresses the limitations of existing systems and achieves significantly better performance in terms of both training throughput and memory consumption.

II. BACKGROUND AND MOTIVATION

A. Message Passing for GNNs

Message passing has become the de-facto programming paradigm for implementing GNN models and is supported by most existing GNN systems [12], [13]. Let $\mathcal{G}(\mathcal{V}, \mathcal{E})$ be a HomoG, where $\mathcal{V}$ and $\mathcal{E}$ are the set of nodes and edges in $\mathcal{G}$, we can define the l -th graph convolution layer in a GNN model using the message passing paradigm as follows:

$$m_e^{(l+1)} = \phi(x_u^{(l)}, x_v^{(l)}, w_e^{(l)}), e = (u, v) \in \mathcal{E}, \quad (1)$$

$$p_v^{(l+1)} = \rho(\{m_e^{(l+1)} : e \in \mathcal{E}\}), \quad (2)$$

$$x_v^{(l+1)} = \psi(x_v^{(l)}, p_v^{(l+1)}), v \in \mathcal{V}, \quad (3)$$

where ϕ is the *message function (MF)* defined on each edge e to generate messages from the feature vector x_u of source node u , the feature x_v of destination node v , and edge feature w_e ; ρ is the *reduce function (RF)* that aggregates the edge-wise messages at the destination node; and ψ is the *update function (UF)* applied to the feature of each destination node.

In practice, existing GNN systems build a connection from the message passing paradigm to sparse matrix operations, e.g., MF to *generalized sampled dense-dense matrix multiplication (g-SDDMM)* and RF to *generalized sparse-dense matrix multiplication (g-SPMM)*.

A HetG is decomposed into multiple homogeneous subgraphs by meta-path based relation decomposition. Then, multiple message passing layers are created for the subgraphs, as illustrated in Fig. 2. Inside a message passing layer, g-SDDMM and g-SPMM are used, which corresponds to the MF and RF in a HomoGNN. Following the same message passing scheme, we obtain multiple embeddings for all types of nodes. Typical HetGNN models apply additional cross-relation aggregation to the output vectors of the RF [14]–[16] to obtain the embedding for each node.

B. Problems of Existing GNN Systems

The programming model and performance optimizations of existing GNN systems are designed for training HomoGNNs. These systems support HetGNN training by simply reusing

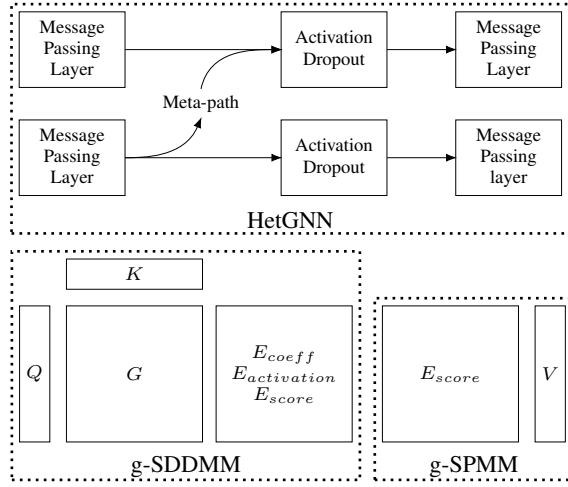


Fig. 2: Message passing and sparse matrix operations in a HetGNN

HomoGNN layers as building blocks. Thus, users need to piece together the HOMO GNN layers decomposed from the HetGNN relations. At runtime, these systems lack a global view of the complete HetGNN computational graph. As a result, these systems can only conduct limited local optimizations constrained in each layer. Their design leads to the following problems.

P1: Large intermediate tensors. Message passing, as a user-friendly programming model, is too high level to realize intermediate results [10], [11]. The number of edges in a graph can be orders of magnitude larger than the number of nodes, which makes messages on edges the memory bottleneck. Fig. 2 (bottom) shows a single message passing layer, where the g-SDDMM operation that computes edge-wise results usually occurs before g-SPMM and generates large intermediate results. Taking GAT as an example, message passing generates intermediate tensors such as edge-wise coefficient E_{coeff} , activated attention $E_{activation}$, and softmax score E_{score} . Each of these tensors has a size of $E \times A$, where E is the number of edges and A is the number of attention heads. Thus, the size of each single intermediate tensor is already at the same magnitude of the entire input graph, making the realization of these intermediate tensors unable to fit in memory.

P2: Fragmented memory usage. The performance of GNN training is also known to be sensitive to memory access [10], [17], [18]. Besides intermediate tensors, models also carry heavy node-wise feature vectors, e.g., Q , K and V in GAT, as shown in Fig. 2. These feature vectors are computed and aggregated at each layer. For HetGNNs, there are multiple message passing layers in one heterogeneous convolution layer. Memory usage is fragmented since existing GNN systems construct separate graph convolution layers for different relations and thus a large number of tensors are created and freed during execution.

P3: Limited operator-wise parallelism. As highlighted in C2 in Section I, HetGNN training exposes much more parallel

execution opportunities than HomoGNN training. However, existing GNN systems interpret users' script as a DAG of GPU tasks, then schedule and execute these tasks sequentially by resolving the dependencies [19]. The number of tasks for HetGNN training increases in proportion to the number of relations. As many groups of tasks share the same data dependencies and a single task usually cannot fully utilize the GPU, there is a large room for parallel execution [20].

III. HOLISTIC REPRESENTATION

The key design of HGL consists of a holistic intermediate representation (HIR) for GNNs and an extensible optimizer that supports both general optimizations and cross-relation optimizations specialized for HetGNNs. We first present HIR in this section and then the optimizations in Section IV.

HIR provides a holistic representation for both HomoGNNs and HetGNNs. GNN models only need to be translated into HIR and transformed to optimized execution plans in the first epoch of training, while subsequent epochs can re-use the result. With the use of HIR, HGL allows users to implement GNN models with the same (familiar) API as in popular GNN systems (e.g., DGL), or migrate existing models to HGL with transparently accelerated model training.

A. The Design of HIR

HIR unifies the representation of HomoGNNs and HetGNNs using a dataflow DAG for the entire model, where nodes in the DAG are operators in HIR that specify the computation for a chunk of data. A group of operators may be organized into a function and then attached to a special operator, while functions can also be nested in functions. This design simplifies the expression for complex models such as GAT. Since GNN models are usually a combination of graph learning and traditional deep learning layers, we define two types of operator primitives, as well as their related attributes, as follows.

Tensor primitives. Tensor primitives take tensors as input and produce tensors as output. HIR simply wraps traditional tensor computations as built-in operators, which simplifies the IR execution by reusing the heavily optimized operator libraries in existing deep learning frameworks.

Graph primitives. Graph primitives are designed for GNNs to allow a direct translation for models implemented using message passing abstractions. To support the graph learning layers implemented using message passing APIs, it suffices to introduce two graph-related operators, *OpNodeFunc* and *OpEdgeFunc*. *OpNodeFunc* defines computation on a single node of the input graph, which takes nodes as input and outputs new node-wise embeddings. *OpEdgeFunc* defines the computation on a single edge and its output. Both operators are attached with a function to perform different user-defined operations. They are expressive since node-wise and edge-wise computations by the MF, RF, and UF (described in Section II-A) can be directly mapped to them. More specifically, *OpEdgeFunc* operates on an edge so that it can access the edge's feature vector and source/destination nodes, and then

store the final results back on the edge, which corresponding to the MF. For the RF and UF, *OpNodeFunc* can access a node with its input edges to receive previous messages from the MF, and perform node-wise *reduce* and *update* operations.

Operator attributes. Operators have attributes such as input/output shape and pointers to learnable parameters. The shape attribute is essential for performing lowering and analyzing fusion opportunities in Section IV. The learnable weights that are defined outside of HIR are traced from the pointers to avoid duplication.

B. HIR Translator

HGL translates GNN models implemented using existing message passing APIs into HIR. The translation begins by dynamic tracing and rebuilding operator dependencies, correctly handling attributes (e.g., shape and learnable weights), and expressing message passing patterns.

Dynamic tracing. In an HIR represented dataflow, operators are inter-connected in a DAG and each operator is related to a single result variable. We use dynamic tracing to construct the dataflow, that is, when a GNN model is declared, we feed a dummy input into the model and trace the model output to rebuild the hidden dependencies.

Saving attributes. During the tracing process, we can visit every intermediate tensor and learn its shape information, which is saved as the attribute of an operator. Besides, differentiable computations may carry learnable weights (e.g., the weight and bias matrices in fully-connected and convolutional layers), which are already allocated before the translation, and hence we also treat them as attributes of an operator and only save their pointers.

Mapping message passing. There are two types of dependencies when tracing, tensor calculation and message passing. For message passing, our graph primitives can be used. According to Equations (1)-(3) in Section II, common graph computation patterns can be generalized to message passing as the MF, RF and UF. For complex combination of messages, we attach successive computations into a single primitive and later perform vertical fusion on it (Section IV-C). After that, *OpEdgeFunc* and *OpNodeFunc* can be used to express the message computation and replace the message passing API.

Once the translation is finished, many opportunities for optimization are opened up (Section IV). HIR also naturally leads to lower performance loss because unlike existing GNN systems (e.g., DGL) where message passing functions are invoked at every epoch, the graph-related operators *OpNodeFunc* and *OpEdgeFunc* are transformed to pure tensor computations as soon as we perform the shape propagation pass (Section IV-A).

C. HIR Use Cases

To better understand HIR, we use R-GAT [2], [8] as an example to illustrate how HIR expresses GNNs. We use R-GAT because it consists of both edge-wise and node-wise computations along with decomposed relations.

DGL script. Fig. 3 shows a simplified implementation of *nn.GATConv* in DGL. Specifically, DGL’s message passing API expresses the GAT algorithm by defining a series of built-in functions such as *u_add_v* (passing messages from source node set *u* and destination node set *v* to edge *e*) and *u_mul_e* (applying multiplication from source *u* to edge *e*, then passing messages from *e* to destination *v*)¹. Our translator traces from the result *graph.dstnode['v']* and analyzes all the dependencies accordingly.

```
# prepare coefficient
graph.dstnode['q'] = input_embedding z_i
graph.srcnode['k'] = input_embedding z_j
graph.message(u_add_v('k', 'q', 'e'))

# apply tensor-wise activation
graph.edge['coeff'] = leaky_relu(graph.edge['e'])

# compute the softmax score
graph.srcnode['u'] = input_embedding z_j
graph.message(edge_softmax('coeff', 'attn'))
graph.message(u_mul_e('u', 'attn', 'm'))

# result in graph.dstnode['v']
graph.message(aggregate_sum('m', 'v'))
```

Fig. 3: Implementation of *GATConv* in DGL

Node-wise aggregation. Tracing back from the model output, we first show how to express node-wise aggregation by HIR in Fig. 4. The input graph structure is treated as a bipartite graph that consists of source and destination node set *u* and *v*, where each node accesses its incoming connections from *node.in_edges* (line 3) and every edge keeps tracking its end nodes in *edge.src* and *edge.dst* (line 4). HIR warps the feature collection step (lines 4-6) into a function and attaches it to *OpReduce* (line 3), which performs node-wise aggregation.

```
1 // aggregate to destination node set
2 %v.h = OpNodeFunc(node: %v, fn: func() {
3   %h = OpReduce(iter: %node.in_edges, fn: func(edge) {
4     %u = %edge.src
5     %score = %edge.attention
6     return OpMul(a: %score, b: %u.h)
7   })
8   return %h
9 })
```

Fig. 4: Node-wise aggregation by HIR

Edge-wise attention. Now consider the more complex edge-wise attention calculation between source and destination nodes. To enable these expressions, we divide the calculation of softmax attention into three steps as in Fig. 5: first the edge-wise numerator of attention score is computed in *OpEdgeFunc* (lines 1-5), then the denominator is calculated by *OpNodeFunc* (lines 6-8), and finally another *OpEdgeFunc* is performed to scale the softmax score on each edge (lines 9-11).

Decomposed relations. With the graph primitives *OpNodeFunc* and *OpEdgeFunc*, the decomposed relations (e.g., Fig. 1) are represented as normal tensor primitives without

¹<https://docs.dgl.ai/api/python/dgl.function.html>

```

1 %e.numerator = OpEdgeFunc(edge: %e, fn: func() {
2   %coeff = OpAdd(a: %edge.src, b: %edge.dst)
3   %activated = OpLeakyRelu(x: %coeff)
4   return OpExp(x: %activated)
5 })
6 %v.denominator = OpNodeFunc(node: %u, fn: func() {
7   return OpSum(x: %node.in_edges.numerator)
8 })
9 %e.attention = OpEdgeFunc(edge: %e, fn: func() {
10   return OpDiv(a: %e.numerator, b: %v.denominator)
11 })

```

Fig. 5: Edge-wise attention calculation by HIR

requiring special operators. As shown in Fig. 6, the relations *paper_cite_paper* and *author_write_paper* generate two new embeddings, *paper.x_0* and *paper.x_1*, which is then summed up to *paper.x* before feeding to the next layer. Note that the two *OpNodeFunc* operators will not submit tensor allocation twice, since common techniques such as variable reuse can be performed easily.

```

1 // paper_cite_paper relation
2 %paper.x_0 = OpNodeFunc(...)
3 // author_write_paper relation
4 %paper.x_1 = OpNodeFunc(...)
5 // aggregate sum on the same paper destination
6 %paper.x = OpAdd(a: %paper.x_0, b: %paper.x_1)

```

Fig. 6: Decomposed relations in HIR

D. Design Trade-offs

To make HIR simple and expressive, we make the following trade-offs.

Auto-differentiation (AD). Supporting AD is challenging. For better compatibility with the underlying DL frameworks, HIR only represents a differentiable forward computation, that is, transformations and optimizations on IR are performed before AD. This design decision misses out some optimization opportunities that are only applicable on one-side computation.

Backward message-passing. The gradient update step of backward pass requires to compute partial derivative on the reversed graph, as discussed in [13]. Instead of storing both the directed graph and its reversed graph, we only keep the original one and perform row-major visit and column-major update, which lowers the memory footprint of HGL but leads to performance degradation.

Static computational graph. Currently, HGL uses dynamic tracing to construct a static computational graph from the original DL program, which assumes that the computational graph remains the same for a model. A static computational graph is used because (1) most existing GNN models have a static computational pattern (e.g., all the models we present in the experiments), and (2) our work focuses on the unified representation HIR and presents critical optimizations for training HetGNNs and HomoGNNs. We are also aware of works that achieve comparable flexibility of dynamic computational graph while enabling optimizations in static computational graph in deep learning, e.g., [21]–[23].

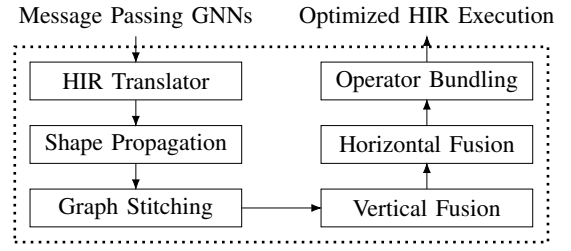


Fig. 7: Lowering and optimization pipeline of HIR

IV. OPTIMIZATIONS

HIR opens up more opportunities for optimizations for HetGNN training. We focus our discussion on the key passes on HIR including one analytical pass and three optimization passes. The optimization pipeline is depicted in Fig. 7. First, we describe the *shape propagation* pass (Section IV-A), which provides the necessary information for optimizations. Second, we introduce *graph stitching* (Section IV-B), which stitches together multiple subgraphs in order to reduce fragmented memory usage. Third, we propose horizontal and vertical *operator fusion* (Section IV-C) to reduce the amount of memory traffic and increase operator-wise parallelism. Finally, we design an *operator bundling* mechanism (Section IV-D) to reduce the number of third-party function calls. We implement these optimizations on top of HIR to address the limitations of existing GNN systems discussed in Section II-B. Note that all these optimizations do not change the semantics of GNN models, and thus the optimized results are mathematically equivalent to the original models.

A. Shape Propagation

Exploiting optimization opportunities over HIR requires a detailed understanding of the input, intermediate, and output tensors. However, existing systems such as DGL and PyG rely on the dynamic computation graph provided by the underlying DL frameworks, making such detailed shape information only accessed and validated at runtime, which limits the potential to further optimize the computation graph in fine granularity.

To enable the optimization of the computation graph, we propose shape propagation to collect detailed tensor shape information at the time of compilation². We demonstrate a shape propagation example of *OpEdgeFunc* in Fig. 8, where *OpEdgeFunc* defines an edge-wise computation that applies activation and exponential functions to the coefficients obtained by aggregating the vectors of the two end nodes of each edge. Here, *func()* is defined on a single edge translated from users' code, specifying that the attention mechanism has 8 heads on this edge. Our system will propagate the shape "[8]" defined on a single edge to the entire graph and pass it to the successor operators.

Moreover, instead of re-collecting and re-calculating such information repeatedly at each epoch as in existing GNN

²Here the time of compilation refers to the phase before the execution of the computation graph.

```

// before
%result: [E, 8] = OpEdgeFunc(edge: %e, fn: func() {
  %coeff: [8] = OpAdd(a: %edge.src, b: %edge.dst)
  %activated: [8] = OpLeakyRelu(x: %coeff)
  return OpExp(x: %activated)
})
// after
%0: [E, 8] = OpSDDMM(...)
%1: [E, 8] = OpLeakyRelu(x: %0)
%result: [E, 8] = OpExp(x: %1)

```

Fig. 8: A shape propagation example of *OpEdgeFunc*

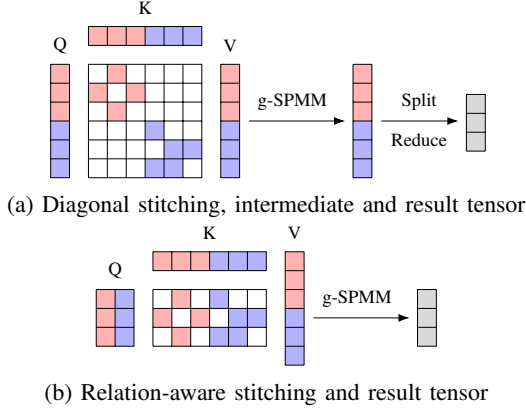


Fig. 9: Comparison of stitching methods

systems (e.g., message passing in DGL is embedded into PyTorch's module and needs to be invoked at each iteration), we collect and validate such information in one pass and reuse it for all the subsequent training epochs (i.e., subsequent execution will directly use the operators with explicit shape, such as *OpSDDMM*[$E, 8$]).

Note that such compile-time collected shape information does not imply that it is unable to handle different graph samples generated during training (e.g., for mini-batch sampling). For runtime-dependent dimensions (i.e., the size of nodes and edges in sampled graphs), we use placeholders (i.e., N and E) as the propagated shape.

B. Graph Stitching

In existing GNN systems, a HetG is decomposed into as many subgraphs (HomoGs) as the number of relations. These HomoGs are represented as sparse adjacency matrices, where node embeddings are represented as dense feature matrices (each row corresponds to the feature of a single node). To learn from the HetG, each relation is calculated independently. However, when the number of relations is large and each subgraph is small, it results in fragmented memory usage and inefficient computation, which further leads to GPU under-utilization. We can reduce the number of small subgraphs by concatenating small graphs together into a large one, as shown in Fig. 9.

First, diagonal stitching can concatenate multiple adjacency matrices along the diagonal of a new matrix and stack the dense feature matrices of nodes, as shown in Fig. 9 (a). How-

ever, such a simple solution suffers from large intermediate tensors, as the dense output tensor of g-SPMM will have the same number of rows as the concatenated adjacency matrix. Moreover, the output will further require an additional split-then-reduce operator to aggregate the embeddings of multiple subgraphs. Thus, such a stitching scheme is not efficient in terms of both memory consumption and computation.

In order to mitigate the issues in diagonal stitching and to stitch subgraphs more efficiently, we propose *relation-aware stitching*. First, HGL groups relations with the same ending type together. Then, the choice of an optimal stitching strategy is formulated into solving a bin-packing problem. After obtaining the stitching strategy, we perform a concatenation for the sparse adjacency matrices. Note that the feature matrices of the ending-type nodes (Q) are concatenated along the feature dimension, while the other feature matrices (K and V) are concatenated along the node dimension, as illustrated in Fig. 9 (b). We re-index the IDs of the nodes/edges in the stitched subgraphs by adding offsets to them, and store the relation index into the higher few bits of each node ID to access its feature in the concatenated feature matrix. With such a stitching method, we no longer need the additional split-then-reduce operation when calculating g-SPMM on the stitched graph. Thus, the issues of large memory consumption and computation overhead in diagonal stitching are resolved.

To achieve the optimal stitching strategy, we formulate it as a bin-packing problem. Given a collection of subgraphs that have the same ending type, we want to pack subgraphs into a minimum number of bins such that the sum of the sizes of the subgraphs to packed into a bin should not exceed its capacity, where the size of a subgraph is the number of edges in the subgraph. The subgraphs in each bin are then stitched together. We use a first-fit-decrease solver to compute the bins, and we prefer an even number of bins because the subsequent optimization techniques such as Horizontal Fusion require at least two stitched subgraphs. Let E be the total number of edges in a collection of subgraphs C and L be the number of edges in the largest subgraph in C . Define $c_i = \max\{\frac{E}{2i}, L\}$, for $1 \leq i < k$ where $\frac{E}{2k} \leq L$. The solver computes the set of bins using each c_i as the bin capacity and then chooses the c_i that gives the set of most balanced packed bins.

To be more specific, the procedure of graph stitching is given as follows:

- 1) Decompose the whole HetG into a set of subgraphs by relations, where each subgraph has only a single type of relation edge.
- 2) Group subgraphs whose relations have the same ending type into a collection.
- 3) For each collection of subgraphs, compute a set of bins by bin-packing.
- 4) For each bin of subgraphs, stitch the subgraphs together by concatenating their graph adjacency matrices along the row dimension, and concatenating other model parameters and input features accordingly. For example, to stitch subgraphs G_1 and G_2 with adjacency matrices $A_1 \in \mathcal{R}_{N \times N_1}$ and $A_2 \in \mathcal{R}_{N \times N_2}$, and node feature

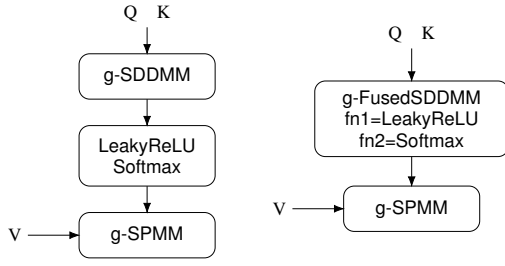


Fig. 10: Vertical fusion

matrices $X_1 \in \mathcal{R}_{N_1 \times F}$ and $X_2 \in \mathcal{R}_{N_2 \times F}$, where N is the number of destination nodes in G_1 and G_2 , N_1 and N_2 are the number of source nodes in G_1 and G_2 respectively, and F is the dimension of the feature vectors, we obtain the concatenated adjacency matrix as $A_1 || A_2$ and the concatenated node feature matrix as $(X_1^T || X_2^T)^T$.

C. Operator Fusion

Operator fusion combines operators into a single kernel. Within the fused kernel, operators can pass results to other operators directly using registers, or execute multiple instructions at one time, thus reducing memory usage and execution latency [24]–[27]. Based on the shape information we retrieve in the previous shape propagation pass, we visit the HIR DAG in topological order to identify fusible operators. We characterize the fusion opportunities into *vertical fusion* and *horizontal fusion* as follows.

Vertical fusion. The objective of vertical fusion is to fuse consecutive operators to their predecessors or successors, and thus intermediate results can be eliminated. Although vertical operator fusion in HomoGNNs is studied in some existing GNN systems such as [11], [13], [28], we propose the first solution to fuse HomoGNNs and HetGNNs operators in a unified manner via HIR.

We perform vertical fusion on most unary operators such as activation function, which helps us address **P1** in Section II-B (i.e., avoiding large intermediate tensor realization). In the motivating example in Fig. 2, message tensors inside message passing layers and tensors after activation dropout layers can avoid being realized if a stack of operators are fused vertically. In order not to introduce extra complexity to the computation DAG, operators with multiple successors are not fused.

Consider the DAG in Fig. 10(left). Our optimizer can fuse a g-SDDMM operation with downstream operators such as LeakyReLU and Softmax into a single operator in Fig. 10(right), which avoids memory traffic caused by reading/writing intermediate tensors from/to limited GPU memory.

Different from the recent FusedMM [28] that fuses the g-SDDMM and g-SPMM operators, we keep them separated in our system for the following reason. As the results of g-SDDMM can be reused in the backward propagation phase, fusing it with g-SPMM will require re-computation in backward propagating. This incurs new computation overhead which we want to avoid.

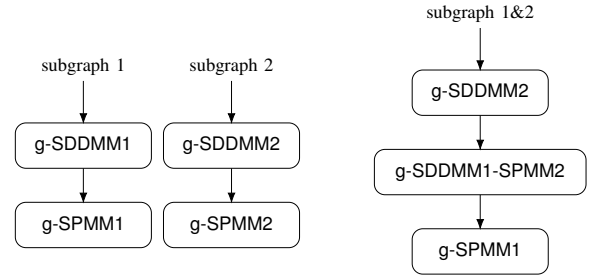


Fig. 11: Horizontal fusion

Horizontal fusion. Horizontal fusion considers operator fusion opportunities across relations. It combines the message passing computation of different relations into one operator. Together with the graph stitching technique in Section IV-B, it addresses the limitation **P3** in Section II-B. Moreover, the fused operator also enjoys better Instruction-Level Parallelism (ILP), thus increasing the GPU utilization.

Typically, for an input adjacency matrix, we launch independent SIMT (single instruction, multiple threads) GPU tasks on each row of the matrix. When two operators have a similar parallel scheme, our optimizer will fuse these operators, which simplifies the computation graph and leads to better instruction-level parallelism. Fig. 11 gives an example where g-SDDMM and g-SPMM for two relations are fused into g-SDDMM, g-SDDMM-SPMM and g-SPMM.

Note that if the optimizer faces equivalent fusion choices among g-SDDMM-SPMM, g-SDDMM-SDDMM and g-SPMM-SPMM, it will take g-SDDMM-SPMM. This is because both g-SDDMM-SDDMM and g-SPMM-SPMM involve symmetric operators only and thus can hardly improve ILP.

D. Operator Bundling

General Matrix Multiplication (GEMM) has become a fundamental building block for high-performance applications. Although GEMM has already been highly optimized by third-party libraries, a large number of external operation invocations may become a bottleneck. In practice, we found that GEMM operation invocations account for about 60% of all the invocations to third-party libraries. To address this problem, we reduce the number of GEMM invocations by simplifying the computation graph using operator bundling.

GEMM operator bundling considers optimizing operator-wise (both intra- and inter-relation) opportunities. Fig. 12 shows an example of intra-relation operator bundling for GEMMs. The g-SDDMM operator in Fig. 12 (a) takes two independent inputs q and k , which invoke the GEMM kernel twice. To reduce GEMM invocations, our optimizer will replace the two GEMM operators with a single BundledGEMM, as shown in Fig. 12 (c). Such bundling opportunities are plentiful in the HIR of typical GNN models. In practice, the number of GEMM kernel calls can be usually reduced by at least $l \times r$, where l and r are the number of layers and relations.

Note that a common function call reduction technique, code-rewriting, is not suitable for HGL. Code-rewriting concate-

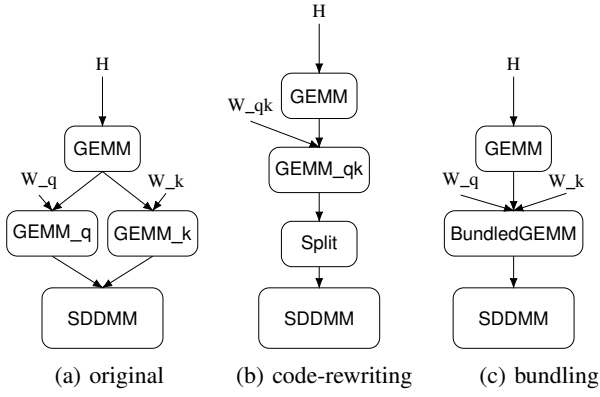


Fig. 12: Operator bundling for GEMMs

mates W_q and W_k to W_{qk} , calls GEMM once to output tensors with double width feature dimensions, and then splits them back, as shown in Fig. 12 (b). As the neural network optimizer (e.g., SGD, Adam) has already stored the references to the learnable W_q and W_k before our HIR is constructed, concatenating these learnable weights at the IR level will invalidate the references of W_q and W_k stored in the neural network optimizer, making the learnable weights W_q and W_k unable to be updated. Thus, though such rewriting can be straightforward on the user script, we are unable to adopt it in our HIR optimization, but use the strided batched bundling described above instead. Such bundling is as effective as code-rewriting in reducing external function calls, while without invalidating the original references to learnable weights.

V. EXPERIMENTAL EVALUATION

We ran all the experiments using single precision floats on Nvidia RTX 3090 (24 GB memory) and RTX 2080 TI (11 GB memory). We compared HGL with two most widely used GNN systems, DGL [13] and PyG [12]. We used the latest versions (at the time of this experimental evaluation), DGL 0.7.x and PyG 2.0.x, and ran all experiments in the same runtime environment (CUDA 11.3 with driver version 465.19.01 was used) for all systems. We evaluated the systems in both full-graph and mini-batch sampling training modes to give a more comprehensive analysis.

We used eight popular datasets as shown in Table I. For full-graph training, we used *cora_tiny* [29], *cora_full* [30], *aifb* [14] and *mutag* [14]. For mini-batch sampling, four larger graphs *amazon* [31], *reddit* [3], *bgs* [14] and *am* [14] were used. Each set of the four datasets consists of two HomoGs and two HetGs.

A. Overall Performance

We first report the overall performance of the systems for training six models, including three most popular HomoGNNs, GCN [8], GAT [2], GraphSAGE [3], and three HetGNNs, R-GCN, R-GAT, and R-GraphSAGE [8], [9]. All results reported are averaged over 20 executions.

Fig. 13 shows that HGL achieves much higher training throughput than both DGL and PyG. Especially for HetGNN

TABLE I: Datasets

graph	type	nodes (types)	edges (relations)	avg. indegree
cora_tiny (CT)	Homo	2.7K (1)	10.6K (1)	3.9
cora_full (CF)	Homo	19.8K (1)	126.8K (1)	31.1
amazon (AZ)	Homo	19.7K (1)	238.2K (1)	6.4
reddit (RD)	Homo	233.0K (1)	114.6M (1)	492.0
aifb (AF)	Het	7.3K (7)	48.8K (104)	4.0
mutag (MT)	Het	27.2K (5)	148.1K (50)	0.6
bgs (BG)	Het	94.8K (27)	672.9K (122)	204.4
am (AM)	Het	1.9M (7)	5.7M (108)	6.5

TABLE II: Memory consumption of PyG, DGL and HGL

model	graph	PyG	DGL	HGL	HGL/DGL
GCN	CT	8.4 MB	2.5 MB	2.3 MB	92.0%
	CF	250.1 MB	43.1 MB	44.0 MB	102.0%
GAT	CT	95.9 MB	42.5 MB	21.7 MB	51.5%
	CF	2.92 GB	1.01 GB	0.53 GB	52.5%
R-GCN	AF	825.5 MB	210.2 MB	30.2 MB	14.4%
	MT	280.4 MB	281.7 MB	54.2 MB	19.2%
R-GAT	AF	11.69 GB	2.49 GB	0.49 GB	19.7%
	MT	3.88 GB	3.26 GB	0.95 GB	29.1%

training, as reported in Fig. 13(e)-(h), HGL’s throughput can be tens to hundreds of times higher than that of DGL and PyG. The performance improvement is mainly because of the native support of HetGNN training enabled by HIR (Section III) and the dedicated optimizations for HetGNN training (Section IV). Although both DGL and PyG are highly optimized for HomoGNN training, HGL still outperforms them, as shown in Fig. 13(a)-(d). This is because HIR allows HGL to exploit more vertical fusion opportunities than DGL and PyG, which only provide a handful fused kernels for built-in operators.

B. Evaluation on Memory Optimization

In this experiment, we evaluate whether HDL’s memory optimization techniques effectively address the problems **P1** and **P2** discussed in Section II-B.

Table II shows that HGL has significantly less memory consumption than DGL and PyG for most datasets. For HomoGNN training, both HGL and DGL use less memory than PyG. Compared with DGL, HGL has similar memory usage for training GCN but uses only half memory of that of DGL when training GAT. For HetGNN training, HGL consumes less memory than both DGL and PyG by a large margin. The improvement is mainly due to the following two optimizations. First, vertical fusion in HGL eliminates intermediate results by fusing consecutive operators. Second, graph stitching eliminates unnecessary tensors in cross-relation aggregation by concatenating small graphs into a large one. We also note that HGL can save more memory for the AF graph than for MT because the effectiveness of vertical fusion is proportional to the number of relations in the graph, and AF has much more relations than MT.

Table III further shows that HGL uses much less small memory blocks (size<1MB) than DGL and PyG. The “reduction” in Table III is the percentage of reduced small memory block

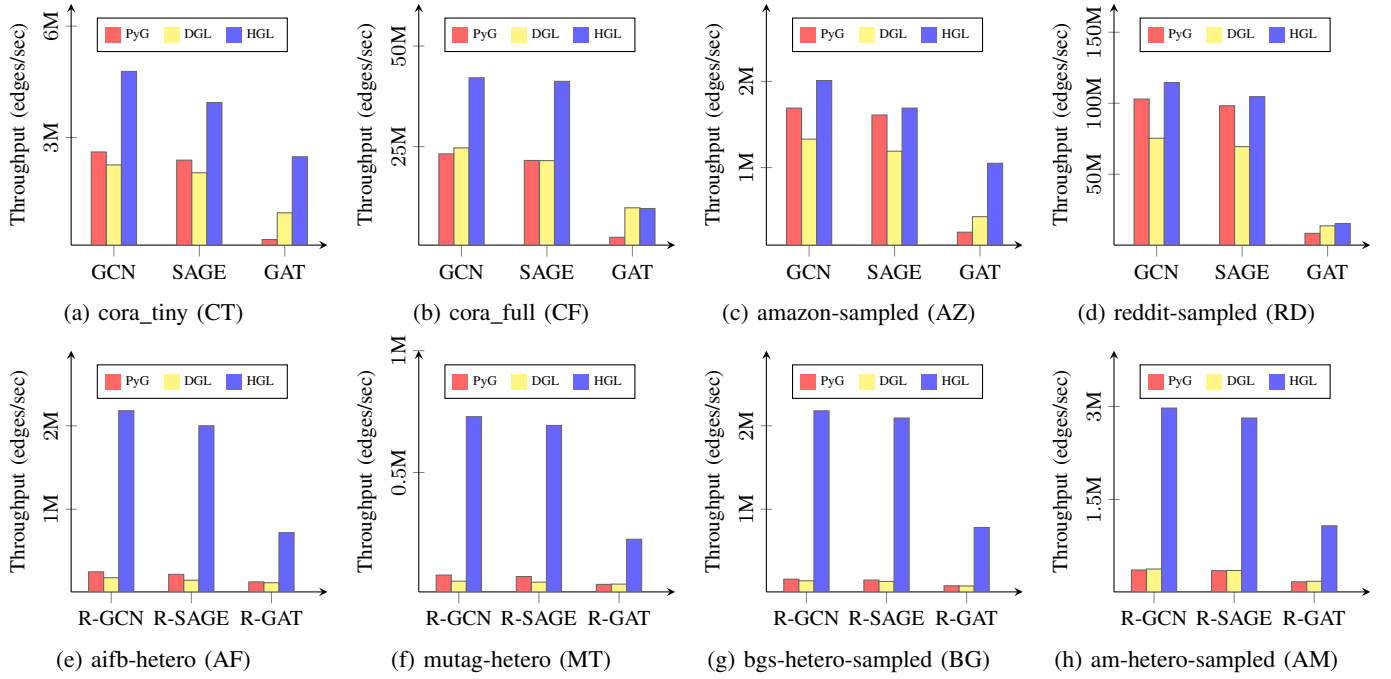


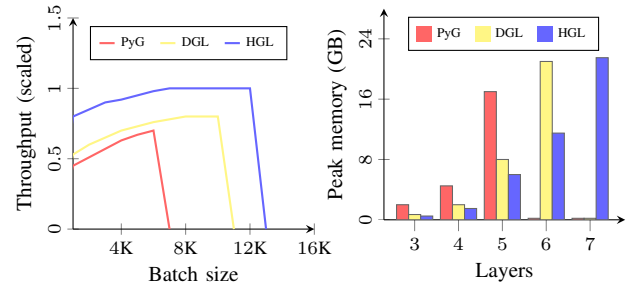
Fig. 13: Training throughput of DGL, PyG and HGL (feature dimension: 32)

TABLE III: Small memory block (<1MB) allocations

model	graph	PyG	DGL	HGL	reduction
GCN	CT	1.7K	640	480	25.0%
	CF	1.3K	360	240	33.3%
GAT	CT	2.2K	1.4K	820	41.4%
	CF	940	860	600	30.2%
R-GCN	AF	151.3K	90.8K	7.3K	92.0%
	MT	71.1K	47.5K	5.6K	88.2%
R-GAT	AF	189.2K	125.4K	30.8K	75.4%
	MT	100.7K	57.0K	14.4K	74.7%

allocations by HGL compared with that by DGL (i.e., (DGL-HGL)/DGL). As allocating many small memory blocks usually leads to fragmented memory usage, the result shows that HGL also effectively addresses **P2** in Section II-B. For HomoGNN training, HGL also allocates less fragmented memory than DGL because vertical fusion eliminates small intermediate results during message passing. For HetGNN training, HGL allocates significantly less fragmented memory blocks than DGL. The improvement mainly comes from graph stitching (concatenating small graphs into a large one) and vertical fusion (fusing consecutive operators), both of which reduce the allocation of small memory blocks.

Reducing memory consumption enables HGL to train with larger batch sizes or deeper models using the same hardware. Fig. 14 (a) shows that HGL's lower memory consumption enables larger batch sizes for GNN training. Fig. 14 (b) shows that when we fix the batch size to 1,024, HGL can train deeper GNN models while PyG and DGL run out-of-memory in training deeper models.



(a) out-of-memory evaluation (b) peak memory evaluation

Fig. 14: Performance with large batches and deeper models

C. Evaluation on Operator Parallelism

This experiment studies if HDL effectively solves the problem **P3** in Section II-B. We show this by measuring the GPU utilization and roofline performance of HGL, DGL and PyG.

Higher GPU utilization. Table IV shows that HGL significantly improves the GPU utilization compared with both PyG and DGL. The reason for the low GPU utilization of DGL and PyG is that they follow users' training script to execute computational kernels sequentially, where each operator cannot highly utilize GPU resources. In comparison, graph stitching, horizontal fusion and operator bundling in HGL combine small workloads into larger ones to make better use of GPU resources. The GPU utilization improvement of R-GAT is higher because R-GAT has more computation-intensive operations (e.g., softmax, leaky_relu activation). Benefited from graph stitching and operator fusion, HGL operates on larger and more intensive fused kernel to utilize GPU more

TABLE IV: Avg GPU utilization and improvement over DGL

model	graph	PyG	DGL	HGL	improvement
R-GCN	AF	11%	7%	15%	114%
	MT	13%	9%	15%	67%
R-GAT	AF	21%	13%	42%	223%
	MT	20%	16%	39%	144%

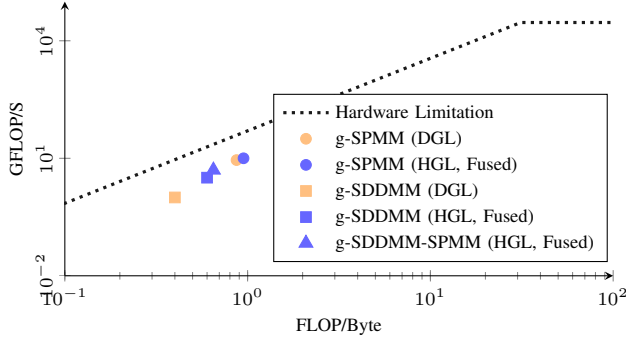


Fig. 15: Achieved roofline performance

TABLE V: Memory saving by individual optimizations

Model	Graph	Graph stitching	Vertical fusion
R-GCN	AF	92%	8%
	MT	87%	13%
R-GAT	AF	17%	83%
	MT	24%	76%

fully, while DGL executes small kernels one by one.

Better kernel performance. We measure the roofline performance [32] of DGL and HGL kernels. The roofline performance of a given model is measured by “operational intensity” instead of traditional arithmetic intensity. We evaluate it by training R-GAT on AF, as R-GAT invokes both g-SPMM and g-SDDMM kernels. The results are picked from their first and second longest running kernels and only consider forward computation. Fig. 15 shows that the g-SPMM kernel in HGL is slightly faster than that in DGL. This is because g-SPMM is the simplest computation and closer to the hardware limitation (i.e., RTX 3090’s single precision floats’ capability). However, HGL’s g-SDDMM (Fused) kernel is much faster than DGL’s g-SDDMM because vertical fusion fuses heavier workloads into a single operator to better utilize the hardware. Lastly, HGL’s g-SDDMM-SPMM (Fused) kernel further improves the performance via horizontal fusion.

D. The Impact of Individual Optimizations

To understand the performance improvements brought by the individual optimization techniques of HGL, we report the memory saving in Table V and the speedup breakdown of the optimization pipeline in Table VI, where *+opt* means *opt* is enabled in addition to the previous optimizations (enabled in the order of HIR, GS, VF, HF, and OB).

For HomoGNN training, vertical fusion and operator bundling are enabled, but performance improvement mainly comes from the former. Operator bundling has less effect

TABLE VI: Speedup breakdown by individual optimizations (GS: graph stitching; VF: vertical fusion; HF: horizontal fusion; OB: operator bundling)

Model	Graph	+HIR	+GS	+VF	+HF	+OB
R-GCN	AF	1.2	12.4	14.2	-	14.6
	MT	1.2	11.6	14.7	-	15.2
	BG (sampled)	1.2	12.9	15.1	-	15.9
	AM (sampled)	1.2	11.4	13.9	-	14.3
R-GAT	AF	1.0	4.5	8.4	8.9	9.2
	MT	1.0	3.3	7.5	7.9	8.1
	BG (sampled)	1.0	3.4	8.7	9.4	9.7
	AM (sampled)	1.0	4.0	7.9	8.5	8.8

because (1) for simple models such as GCN, few operators can be bundled, and (2) for more complex models such as GAT, the bottlenecks are graph-related computation and intermediate memory usage, which is more effectively solved by vertical fusion.

For HetGNN training, different optimizations are tightly coupled to bring the performance improvements for different graph structures and models. From the reported results, we have the following observations:

- Once a HetGNN model is transformed to HIR, it naturally achieves a small amount of speedup as a result of eliminating costly message passing calls, although this benefit is negligible for the computationally intensive R-GAT model.
- Graph stitching contributes to a significant portion of the performance improvements. One reason is that graph stitching provides efficient data structure (i.e., the relation-aware stitched graph). R-GCN, the less computational intensive model, obtains more benefits from graph stitching (e.g., 85% of the total speedup on AF and 76% on MT) because subgraphs with a small number of corresponding nodes can become stragglers and slow down the whole training process. The improvement brought by graph stitching also shows a positive correlation to the number of relations.
- Operator fusion, especially vertical fusion, is more effective for training R-GAT (e.g., it produces another $1.8\times$ speedup on AF and $3.1\times$ on MT) for its effectiveness of reducing read and write latency on a large amount of intermediate results. Improvement brought by horizontal fusion are incremental, as both g-SDDMM and g-SPMM are memory-bound operators (i.e., on the left side of the roofline diagram), which may reach the hardware limit.
- Operator bundling further improves the performance, but the benefit of reducing GEMM function calls is not as obvious as that of the other optimizations.
- Vertical fusion saves more memory for R-GAT by eliminating intermediate results from contiguous operators (e.g., softmax and leaky_relu activation). However, R-GCN has less fusible operators and its memory saving mainly comes from graph stitching.

TABLE VII: Normalized training throughput (DGL / HGL / speedup), feature dimension: 16/32/64

graph	GPU	GCN for CT&CF / R-GCN for AF&MT			GAT for CT&CF / R-GAT for AF&MT		
		16	32	64	16	32	64
CT	RTX 2080 TI	0.96 / 1.92 / 2.00	0.89 / 1.99 / 2.24	0.97 / 1.94 / 2.00	0.99 / 2.61 / 2.64	0.99 / 2.60 / 2.63	0.99 / 2.62 / 2.65
	RTX 3090	1.00 / 1.87 / 1.87	0.87 / 1.93 / 2.22	0.98 / 1.85 / 1.89	1.00 / 2.63 / 2.63	0.99 / 2.67 / 2.70	0.96 / 2.62 / 2.73
CF	RTX 2080 TI	0.67 / 0.72 / 1.07	0.67 / 0.72 / 1.07	0.51 / 0.55 / 1.06	0.66 / 0.63 / 0.95	0.51 / 0.52 / 1.02	0.30 / 0.29 / 0.97
	RTX 3090	1.00 / 1.71 / 1.71	0.90 / 1.55 / 1.72	0.84 / 1.24 / 1.48	1.00 / 1.00 / 1.00	0.78 / 0.79 / 1.01	0.64 / 0.64 / 1.01
AF	RTX 2080 TI	1.06 / 15.87 / 14.97	0.95 / 15.04 / 15.83	0.99 / 14.84 / 14.99	1.02 / 8.40 / 8.33	0.94 / 9.88 / 10.51	0.96 / 8.77 / 9.14
	RTX 3090	1.00 / 22.36 / 22.36	1.03 / 15.04 / 14.60	0.90 / 16.07 / 15.03	1.00 / 8.81 / 8.81	0.98 / 8.98 / 9.16	0.93 / 8.34 / 8.97
MT	RTX 2080 TI	0.93 / 15.74 / 16.92	0.91 / 13.36 / 14.68	0.88 / 13.13 / 14.92	0.91 / 8.04 / 8.84	0.93 / 6.60 / 7.10	0.88 / 7.85 / 8.92
	RTX 3090	1.0 / 12.50 / 12.50	0.89 / 13.51 / 15.18	0.96 / 12.41 / 12.93	1.0 / 7.22 / 7.22	0.94 / 7.58 / 8.06	0.92 / 7.83 / 8.51

TABLE VIII: Baseline throughput (edges/sec) on RTX 3090

Model	Graph	DGL	HGL
GCN	CT	2.3M	4.3M
	CF	25.1M	42.8M
GAT	CT	1.0M	2.6M
	CF	10.0M	10.0M
R-GCN	AF	116.3K	2.6M
	MT	56.2K	0.7M
R-GAT	AF	76.7K	676.1K
	MT	35.6K	256.8K

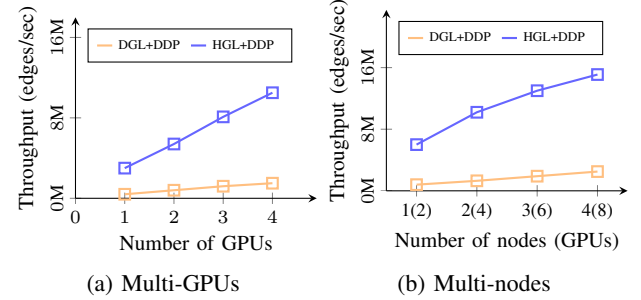


Fig. 16: Scalability of DGL and HGL

E. Different Hardware and Hyperparameter

We also evaluated HGL on different hardware and hyperparameter configurations to demonstrate the robustness of HGL's optimization techniques. Table VII shows that HGL achieves competitive (from similar to more than twice better) performance compared with DGL for training GCN and GAT on the CT and CF datasets, but HGL's throughput is significantly higher than that of DGL for training HetGNNs. The performance speedups are also consistent with different GPU configurations and different feature dimensions. The baseline throughput of Table VII is given in Table VIII.

F. Scalability

We evaluated the scalability of HGL by implementing distributed data-parallel (DDP) training with synchronous SGD on multi-GPU and multi-node settings. We used DGL as a baseline, and HGL and DGL used the same strategies for data sampling and data movement in the experiment. We also used the same hyperparameter settings as in Fig. 13. Fig. 16 reports the performance results of training R-GAT on the larger graph AM, which show that HGL achieves near linear scalability on both multi-GPU and multi-node settings. Similar results were also observed for other datasets and training other HetGNNs (omitted due to the page limit).

VI. RELATED WORK

Heterogeneous GNN training. DGL [13] and PyG [12] are the most popular GNN systems. They have built libraries for users to query and manipulate HetGs and perform message-passing operations on HetGs. However, both DGL and PyG lack a unified intermediate representation for GNNs, which

imposes a great limitation for holistic optimizations for HetGNN training that we support. DGL has manually optimized their R-GCN implementation but it remains unclear how to extend such optimization to general HetGNNs.

Kernel optimizations for GNNs. The irregular memory access pattern caused by graph data makes it challenging to design efficient kernels for GNNs. Many systems have been proposed and they differ in the designs of a variety of optimizations [10], [11], [28], [33]–[38]. The optimizations can be roughly characterized into dataflow optimizations (e.g., operator fusion [11], [28], [37]), graph data optimizations (e.g., node re-ordering to improve locality [10], [35]), and hardware-specific optimizations (e.g., tensor core acceleration [38]). However, these optimizations are designed for HomoGNNs and optimization opportunities for HetGNNs are not explored.

Scalable GNN training. Partitioned training partitions a large graph so that each partition can fit into memory of a single device and multiple workers work on individual partitions in parallel. Various partitioning and pipelining strategies have been proposed to hide the latency of data movement [39]–[44]. There are also works focusing on mitigating the data communication bottleneck by reducing the amount of data traffic [45] or automatically devising communication scheduling schemes for heterogeneous data communication topology [46]. P^3 proposes a pipelined push-pull parallelism strategy to accelerate distributed GNN training [47]. Sampling-based training [48]–[51] constructs a mini-batch of samples by sampling a subgraph that can fit into memory of a single device and performs data-parallel training as traditional DL models. They propose various techniques to accelerate training such as

GPU-based data sampling, data pre-fetching and pipelining.

VII. CONCLUSIONS

We designed HGL that optimizes HetGNN training by diving into the characteristics of memory usage and computation pattern in HetGNN training. Our experiments verify that HGL significantly reduces both overall and fragmented memory usage, while increasing GPU utilization by exploiting more parallelism brought by HIR.

ACKNOWLEDGMENT

The authors would like to thank the reviewers for their valuable comments and suggestions that have helped significantly improve the paper.

REFERENCES

- [1] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017. [Online]. Available: <https://openreview.net/forum?id=SJU4ayYgl>
- [2] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net, 2018. [Online]. Available: <https://openreview.net/forum?id=rJXMpikCZ>
- [3] W. L. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, I. Guyon, U. von Luxburg, S. Bengio, H. M. Wallach, R. Fergus, S. V. N. Vishwanathan, and R. Garnett, Eds., 2017, pp. 1024–1034.
- [4] C. Zhang, D. Song, C. Huang, A. Swami, and N. V. Chawla, "Heterogeneous graph neural network," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ser. KDD '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 793–803. [Online]. Available: <https://doi.org/10.1145/3292500.3330961>
- [5] Z. Liu, B. Ma, Q. Liu, J. Xu, and B. Zheng, *Heterogeneous Graph Neural Networks for Large-Scale Bid Keyword Matching*. New York, NY, USA: Association for Computing Machinery, 2021, p. 3976–3985. [Online]. Available: <https://doi.org/10.1145/3459637.3481926>
- [6] C. Shi, Y. Li, J. Zhang, Y. Sun, and P. S. Yu, "A survey of heterogeneous information network analysis," *IEEE Trans. on Knowl. and Data Eng.*, vol. 29, no. 1, p. 17–37, jan 2017. [Online]. Available: <https://doi.org/10.1109/TKDE.2016.2598561>
- [7] J. Xu, Z. Zhu, J. Zhao, X. Liu, M. Shan, and J. Guo, *Gemini: A Novel and Universal Heterogeneous Graph Information Fusing Framework for Online Recommendations*. New York, NY, USA: Association for Computing Machinery, 2020, p. 3356–3365. [Online]. Available: <https://doi.org/10.1145/3394486.3403388>
- [8] P. W. Battaglia, J. B. Hamrick, V. Bapst, A. Sanchez-Gonzalez, V. F. Zambaldi, M. Malinowski, A. Tacchetti, D. Raposo, A. Santoro, R. Faulkner, Ç. Gülçehre, H. F. Song, A. J. Ballard, J. Gilmer, G. E. Dahl, A. Vaswani, K. R. Allen, C. Nash, V. Langston, C. Dyer, N. Heess, D. Wierstra, P. Kohli, M. Botvinick, O. Vinyals, Y. Li, and R. Pascanu, "Relational inductive biases, deep learning, and graph networks," *CoRR*, vol. abs/1806.01261, 2018. [Online]. Available: <http://arxiv.org/abs/1806.01261>
- [9] W. Hu, M. Fey, H. Ren, M. Nakata, Y. Dong, and J. Leskovec, "Ogb-lsc: A large-scale challenge for machine learning on graphs," 2021.
- [10] K. Huang, J. Zhai, Z. Zheng, Y. Yi, and X. Shen, *Understanding and Bridging the Gaps in Current GNN Performance Optimizations*. New York, NY, USA: Association for Computing Machinery, 2021, p. 119–132. [Online]. Available: <https://doi.org/10.1145/3437801.3441585>
- [11] Y. Wu, K. Ma, Z. Cai, T. Jin, B. Li, C. Zheng, J. Cheng, and F. Yu, "Seastar: Vertex-centric programming for graph neural networks," in *Proceedings of the Sixteenth European Conference on Computer Systems*, ser. EuroSys '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 359–375. [Online]. Available: <https://doi.org/10.1145/3447786.3456247>
- [12] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- [13] M. Wang, D. Zheng, Z. Ye, Q. Gan, M. Li, X. Song, J. Zhou, C. Ma, L. Yu, Y. Gai, T. Xiao, T. He, G. Karypis, J. Li, and Z. Zhang, "Deep graph library: A graph-centric, highly-performant package for graph neural networks," 2020.
- [14] M. S. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling, "Modeling relational data with graph convolutional networks," in *The Semantic Web - 15th International Conference, ESWC 2018, Heraklion, Crete, Greece, June 3-7, 2018, Proceedings*, ser. Lecture Notes in Computer Science, A. Gangemi, R. Navigli, M. Vidal, P. Hitzler, R. Troncy, L. Hollink, A. Tordai, and M. Alam, Eds., vol. 10843. Springer, 2018, pp. 593–607. [Online]. Available: https://doi.org/10.1007/978-3-319-93417-4_38
- [15] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, and P. S. Yu, "Heterogeneous graph attention network," in *The World Wide Web Conference, WWW 2019, San Francisco, CA, USA, May 13-17, 2019*, L. Liu, R. W. White, A. Mantrach, F. Silvestri, J. J. McAuley, R. Baeza-Yates, and L. Zia, Eds. ACM, 2019, pp. 2022–2032. [Online]. Available: <https://doi.org/10.1145/3308558.3313562>
- [16] Z. Hu, Y. Dong, K. Wang, and Y. Sun, "Heterogeneous graph transformer," in *WWW '20: The Web Conference 2020, Taipei, Taiwan, April 20-24, 2020*, Y. Huang, I. King, T. Liu, and M. van Steen, Eds. ACM / IW3C2, 2020, pp. 2704–2710. [Online]. Available: <https://doi.org/10.1145/3366423.3380027>
- [17] M. Winter, D. Mlakar, R. Zayer, H.-P. Seidel, and M. Steinberger, "Adaptive sparse matrix-matrix multiplication on the gpu," in *Proceedings of the 24th Symposium on Principles and Practice of Parallel Programming*, ser. PPOPP '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 68–81. [Online]. Available: <https://doi.org/10.1145/3293883.3295701>
- [18] M. Parger, M. Winter, D. Mlakar, and M. Steinberger, *SpECK: Accelerating GPU Sparse Matrix-Matrix Multiplication through Lightweight Analysis*. New York, NY, USA: Association for Computing Machinery, 2020, p. 362–375. [Online]. Available: <https://doi.org/10.1145/3332466.3374521>
- [19] W. Kwon, G. Yu, E. Jeong, and B. Chun, "Nimble: Lightweight and parallel GPU task scheduling for deep learning," in *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., 2020.
- [20] Y. Ding, L. Zhu, Z. Jia, G. Pekhimenko, and S. Han, "Ios: Inter-operator scheduler for cnn acceleration," 2020. [Online]. Available: <https://arxiv.org/abs/2011.01302>
- [21] E. Jeong, S. Cho, G. Yu, J. S. Jeong, D. Shin, and B. Chun, "JANUS: fast and flexible deep learning via symbolic graph execution of imperative programs," in *16th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2019, Boston, MA, February 26-28, 2019*, J. R. Lorch and M. Yu, Eds. USENIX Association, 2019, pp. 453–468. [Online]. Available: <https://www.usenix.org/conference/nsdi19/presentation/jeong>
- [22] Y. Yu, M. Abadi, P. Barham, E. Brevdo, M. Burrows, A. Davis, J. Dean, S. Ghemawat, T. Harley, P. Hawkins, M. Isard, M. Kudlur, R. Monga, D. G. Murray, and X. Zheng, "Dynamic control flow in large-scale machine learning," in *Proceedings of the Thirteenth EuroSys Conference, EuroSys 2018, Porto, Portugal, April 23-26, 2018*, R. Oliveira, P. Felber, and Y. C. Hu, Eds. ACM, 2018, pp. 18:1–18:15. [Online]. Available: <https://doi.org/10.1145/3190508.3190551>
- [23] K. Zhu, W. Zhao, Z. Zheng, T. Guo, P. Zhao, J. Bai, J. Yang, X. Liu, L. Diao, and W. Lin, "DISC: A dynamic shape compiler for machine learning workloads," in *EuroMLSys@EuroSys 2021, Proceedings of the 1st Workshop on Machine Learning and Systems Virtual Event, Edinburgh, Scotland, UK, 26 April, 2021*, E. Yoneki and P. Patras, Eds. ACM, 2021, pp. 89–95. [Online]. Available: <https://doi.org/10.1145/3437984.3458838>

- [24] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, M. Cowan, H. Shen, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "Tvm: An automated end-to-end optimizing compiler for deep learning," in *Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI'18. USA: USENIX Association, 2018, p. 579–594.
- [25] A. Li, B. Zheng, G. Pekhimenko, and F. Long, "Automatic horizontal fusion for GPU kernels," in *IEEE/ACM International Symposium on Code Generation and Optimization, CGO 2022, Seoul, Korea, Republic of, April 2-6, 2022*, J. W. Lee, S. Hack, and T. Shpeisman, Eds. IEEE, 2022, pp. 14–27. [Online]. Available: <https://doi.org/10.1109/CGO53902.2022.9741270>
- [26] P. Sarthi, K. Rajan, A. Lal, A. Modi, P. Jain, M. Liu, A. Gosalia, and S. Kalikar, *Generalized Sub-Query Fusion for Eliminating Redundant I/O from Big-Data Queries*. USA: USENIX Association, 2020.
- [27] A. Ivanov, N. Dryden, T. Ben-Nun, S. Li, and T. Hoefler, "Data movement is all you need: A case study on optimizing transformers," 2020. [Online]. Available: <https://arxiv.org/abs/2007.00072>
- [28] M. K. Rahman, M. H. Sujon, and A. Azad, "Fusedmm: A unified sddmm-spm kernel for graph embedding and graph neural networks," in *35th IEEE International Parallel and Distributed Processing Symposium, IPDPS 2021, Portland, OR, USA, May 17-21, 2021*. IEEE, 2021, pp. 256–266. [Online]. Available: <https://doi.org/10.1109/IPDPS49936.2021.00034>
- [29] Z. Yang, W. W. Cohen, and R. Salakhutdinov, "Revisiting semi-supervised learning with graph embeddings," in *Proceedings of the 33rd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016*, ser. JMLR Workshop and Conference Proceedings, M. Balcan and K. Q. Weinberger, Eds., vol. 48. JMLR.org, 2016, pp. 40–48. [Online]. Available: <http://proceedings.mlr.press/v48/yang16.html>
- [30] A. Bojchevski and S. Günnemann, "Deep gaussian embedding of graphs: Unsupervised inductive learning via ranking," in *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net, 2018. [Online]. Available: <https://openreview.net/forum?id=r1ZdKJ-0W>
- [31] O. Shchur, M. Mumme, A. Bojchevski, and S. Günnemann, "Pitfalls of graph neural network evaluation," *CoRR*, vol. abs/1811.05868, 2018. [Online]. Available: <http://arxiv.org/abs/1811.05868>
- [32] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, p. 65–76, apr 2009. [Online]. Available: <https://doi.org/10.1145/1498765.1498785>
- [33] Y. Wu, Y. Gui, T. Jin, J. Cheng, X. Yan, P. Yin, Y. Cai, B. Tang, and F. Yu, "Vertex-centric visual programming for graph neural networks," in *SIGMOD '21: International Conference on Management of Data, Virtual Event, China, June 20-25, 2021*, G. Li, Z. Li, S. Idreos, and D. Srivastava, Eds. ACM, 2021, pp. 2803–2807. [Online]. Available: <https://doi.org/10.1145/3448016.3452770>
- [34] Y. Hu, Z. Ye, M. Wang, J. Yu, D. Zheng, M. Li, Z. Zhang, Z. Zhang, and Y. Wang, "Featgraph: a flexible and efficient backend for graph neural network systems," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2020, Virtual Event / Atlanta, Georgia, USA, November 9-19, 2020*, C. Cuicchi, I. Qualters, and W. T. Kramer, Eds. IEEE/ACM, 2020, p. 71. [Online]. Available: <https://doi.org/10.1109/SC41405.2020.00075>
- [35] Y. Wang, B. Feng, G. Li, S. Li, L. Deng, Y. Xie, and Y. Ding, "Gnnadvisor: An adaptive and efficient runtime system for GNN acceleration on gpus," in *15th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2021, July 14-16, 2021*, A. D. Brown and J. R. Lorch, Eds. USENIX Association, 2021, pp. 515–531. [Online]. Available: <https://www.usenix.org/conference/osdi21/presentation/wang-yuke>
- [36] G. Huang, G. Dai, Y. Wang, and H. Yang, "Ge-spm: general-purpose sparse matrix-matrix multiplication on gpus for graph neural networks," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2020, Virtual Event / Atlanta, Georgia, USA, November 9-19, 2020*, C. Cuicchi, I. Qualters, and W. T. Kramer, Eds. IEEE/ACM, 2020, p. 72. [Online]. Available: <https://doi.org/10.1109/SC41405.2020.00076>
- [37] H. Liu, S. Lu, X. Chen, and B. He, "G3: when graph neural networks meet parallel graph processing systems on gpus," *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 2813–2816, 2020. [Online]. Available: <http://www.vldb.org/pvldb/vol13/p2813-liu.pdf>
- [38] Y. Wang, B. Feng, and Y. Ding, "Qgtc: Accelerating quantized gnn via gpu tensor core," *arXiv preprint arXiv:2111.09547*, 2021.
- [39] L. Ma, Z. Yang, Y. Miao, J. Xue, M. Wu, L. Zhou, and Y. Dai, "Neugraph: Parallel deep neural network computation on large graphs," in *2019 USENIX Annual Technical Conference, USENIX ATC 2019, Renton, WA, USA, July 10-12, 2019*, D. Malkhi and D. Tsafir, Eds. USENIX Association, 2019, pp. 443–458. [Online]. Available: <https://www.usenix.org/conference/atc19/presentation/ma>
- [40] C. Li, Y. Wang, C. Liu, S. Liang, H. Li, and X. Li, "GLIST: towards in-storage graph learning," in *2021 USENIX Annual Technical Conference, USENIX ATC 2021, July 14-16, 2021*, I. Calciu and G. Kuenning, Eds. USENIX Association, 2021, pp. 225–238. [Online]. Available: <https://www.usenix.org/conference/atc21/presentation/li-cangyuan>
- [41] L. Wang, Q. Yin, C. Tian, J. Yang, R. Chen, W. Yu, Z. Yao, and J. Zhou, "Flexgraph: a flexible and efficient distributed framework for GNN training," in *EuroSys '21: Sixteenth European Conference on Computer Systems, Online Event, United Kingdom, April 26-28, 2021*, A. Barbalace, P. Bhatotia, L. Alvisi, and C. Cadar, Eds. ACM, 2021, pp. 67–82. [Online]. Available: <https://doi.org/10.1145/3447786.3456229>
- [42] M. Tanaka, K. Taura, T. Hanawa, and K. Torisawa, "Automatic graph partitioning for very large-scale deep learning," in *35th IEEE International Parallel and Distributed Processing Symposium, IPDPS 2021, Portland, OR, USA, May 17-21, 2021*. IEEE, 2021, pp. 1004–1013. [Online]. Available: <https://doi.org/10.1109/IPDPS49936.2021.00109>
- [43] Z. Jia, S. Lin, M. Gao, M. Zaharia, and A. Aiken, "Improving the accuracy, scalability, and performance of graph neural networks with roc," in *Proceedings of Machine Learning and Systems 2020, MLSys 2020, Austin, TX, USA, March 2-4, 2020*, I. S. Dhillon, D. S. Papailiopoulou, and V. Sze, Eds. mlsys.org, 2020. [Online]. Available: <https://proceedings.mlsys.org/book/300.pdf>
- [44] J. Mohoney, R. Waleffe, H. Xu, T. Rekatsinas, and S. Venkataraman, "Marius: Learning massive graph embeddings on a single machine," in *15th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2021, July 14-16, 2021*, A. D. Brown and J. R. Lorch, Eds. USENIX Association, 2021, pp. 533–549. [Online]. Available: <https://www.usenix.org/conference/osdi21/presentation/mohoney>
- [45] A. Tripathy, K. A. Yelick, and A. Buluç, "Reducing communication in graph neural network training," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2020, Virtual Event / Atlanta, Georgia, USA, November 9-19, 2020*, C. Cuicchi, I. Qualters, and W. T. Kramer, Eds. IEEE/ACM, 2020, p. 70. [Online]. Available: <https://doi.org/10.1109/SC41405.2020.00074>
- [46] Z. Cai, X. Yan, Y. Wu, K. Ma, J. Cheng, and F. Yu, "DGCL: an efficient communication library for distributed GNN training," in *EuroSys '21: Sixteenth European Conference on Computer Systems, Online Event, United Kingdom, April 26-28, 2021*, A. Barbalace, P. Bhatotia, L. Alvisi, and C. Cadar, Eds. ACM, 2021, pp. 130–144. [Online]. Available: <https://doi.org/10.1145/3447786.3456233>
- [47] S. Gandhi and A. P. Iyer, "P3: distributed deep graph learning at scale," in *15th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2021, July 14-16, 2021*, A. D. Brown and J. R. Lorch, Eds. USENIX Association, 2021, pp. 551–568. [Online]. Available: <https://www.usenix.org/conference/osdi21/presentation/gandhi>
- [48] A. Jangda, S. Polisetty, A. Guha, and M. Serafini, "Accelerating graph sampling for graph machine learning using gpus," 2021.
- [49] D. Zheng, C. Ma, M. Wang, J. Zhou, Q. Su, X. Song, Q. Gan, Z. Zhang, and G. Karypis, "Distdgl: Distributed graph neural network training for billion-scale graphs," in *10th IEEE/ACM Workshop on Irregular Applications: Architectures and Algorithms, IA3 2020, Atlanta, GA, USA, November 11, 2020*. IEEE, 2020, pp. 36–44. [Online]. Available: <https://doi.org/10.1109/IA351965.2020.00011>
- [50] T. Kaler, N. Stathas, A. Ouyang, A.-S. Iliopoulos, T. B. Schardl, C. E. Leiserson, and J. Chen, "Accelerating training and inference of graph neural networks with fast sampling and pipelining," *arXiv preprint arXiv:2110.08450*, 2021.
- [51] C. Zheng, H. Chen, Y. Cheng, Z. Song, Y. Wu, C. Li, J. Cheng, H. Yang, and S. Zhang, "Bytegnn: Efficient graph neural network training at large scale," *Proc. VLDB Endow.*, vol. 15, no. 6, pp. 1228–1242, 2022. [Online]. Available: <https://www.vldb.org/pvldb/vol15/p1228-zheng.pdf>

Appendix: Artifact Description/Artifact Evaluation

SUMMARY OF THE EXPERIMENTS REPORTED

0.1 Platform Setup

Hardware. We conducted all the experiments on RTX 3090 (24 GB memory) and RTX 2080 TI (11 GB memory) GPUs.

Software. The required software are listed in Table 1.

Table 1: Software versions

name	version	build
Ubuntu	18.04	-
Kernel	4.15.0-143-generic	buildd@lcy01-amd64-001
GCC	7.5.0	-
GPU-Driver	465.19.01	-
CUDA-Toolkit	11.3	-
Python	3.8.12	h12debd9_0
PyTorch	1.10.2	py3.8_cuda11.3_cudnn8.2.0_0
DGL	0.7.2	dgl-cuda11.3
PyG	2.0.3	py38_torch_1.10.0_cu113

0.2 Experiment Setup

Build Essentials. Install build essentials, *build-essential*, *python3-setuptools*, and *ninja-build* are required.

Installing CUDA. Install CUDA 11.3 and compatible drivers from <https://developer.nvidia.com/cuda-downloads>

Installing Conda. Install Anaconda or Miniconda from <https://docs.conda.io>

Creating Python Environment. Create a new clean environment with “*conda create -n hgl python=3.8 && source activate hgl*”

Installing dependencies, baseline systems, and HGL.

- PyTorch: *conda install pytorch=1.10.2 cudatoolkit=11.3 -c pytorch -c conda-forge*
- DGL: *conda install dgl-cuda11.3=0.7.2 -c dglteam -c conda-forge*
- RDFLib: *pip3 install rdflib*
- HGL: *cd hgl-proto && python3 setup.py install*
- PyG (not required): *conda install pyg=2.0.1 -c pyg -c conda-forge*

Note that HGL reuses datasets from DGL, which relies on the *RDFLib* tool, and the installation of PyG is not required.

Automatic Diagnostic. Users can check if their system configurations are compatible with HGL by running “*python3 setup.py test*”, which performs hardware diagnostic and result checking for HGL.

0.3 Correctness of HGL

We provide 5 test cases to make sure HGL works as expected:

- *test_kernel.py*: checking CUDA kernel results (e.g., fused g-SPMM and g-SDDMM) with equivalent dense matrix computations.

```
# select one GPU
GPU=0
# print dataset information
PYTHONPATH=. python3 test/bench_macro.py --info
# benchmark on different settings
for lib in 'pyg' 'dgl' 'hgl'; do
  for model in 'rgcn' 'rgat'; do
    for dataset in 'aifb_hetero' 'mutag_hetero'; do
      for d_hidden in 16 32 64; do
        echo $lib $model $dataset $d_hidden
        PYTHONPATH=. CUDA_VISIBLE_DEVICES=$GPU \
        python3 test/bench_macro.py \
        --lib=$lib \
        --model=$model \
        --dataset=$dataset \
        --d_hidden=$d_hidden
      done
    done
  done
done
```

Figure 1: The script to reproduce and measure both memory and computational metrics.

- *test_allclose.py*: checking GNN results by layer between HGL and DGL, that makes sure HGL does not miss calculations.
- *test_homo.py*: checking convergence of GAT model by training a minimal homogeneous model on HGL.
- *test_homo.py*: checking convergence of R-GAT model by training a minimal homogeneous model HGL.
- *test_stitch.py*: checking the correctness of graph stitching by comparing results when stitching is enabled and disabled.

0.4 Running Experiments

We provide a python script *hgl-proto/bench_macro.py* to enable an easy reproducibility process. The script takes 4 arguments:

- *-lib*: specifying which implementation to use, which can be *pyg*, *dgl*, or *hgl* (required)
- *-model*: choosing the GNN model to evaluate, which can be *gcn*, *sage*, *gat*, *rgcn*, *rsage*, or *rgat* (required)
- *-dataset*: selecting the graph dataset, which can be *cora_tiny*, *cora_full*, *amazon*, *reddit*, *aifb_hetero*, *mutag_hetero*, *bgs_hetero*, or *am_hetero* (required)
- *-d_hidden*: specifying the hidden dimension size of the model, which can be any positive integer number (required)

The output is printed in the *stdout* stream, which includes all important metrics and stats (allocated memory size and count, small and large memory blocks, and throughput) as well as time measurements.

One script for all. Fig. 1 shows a script to reproduce the results reported in Section V.B and Section V.E. It first configures the GPU slot and print the dataset information, and then launches each test as a separate process, such that failed tasks (e.g., wrong settings and out-of-memory) do not impede the launches after that.

Memory performance reported in Section V.B. The results are calculated on top of PyTorch’s buffer pool, so that optimizations by the buffer pool and GPU driver are ignored.

Throughput performance reported in Section V.A and Section V.E. The throughput measurements in output are averaged over 20 executions, where abnormal results are removed.

AUTHOR-CREATED OR MODIFIED ARTIFACTS:

Artifact 1

Persistent ID: <https://doi.org/10.5281/zenodo.6914329>

Artifact name: HGL prototype implementation

Reproduction of the artifact without container: Packing close source GPU drivers with compiled kernels into container may cause unexpected behavior and errors. Instead, we give step by step installation guide as aforementioned.